`



Java Institute for Advanced Technology

Department of Examinations

COURSE(S) - LEADING TO	BSC (HONS) IN SOFTWARE ENGINEERING
UNIT NAME	HANDHELD DEVICE PROGRAMMING II
UNIT ID	JIAT / HHDPII
EXAMINATION ID	JIAT / HHDPII / EX01

NAME : H.M. ATHULYA HERATH

ADMISSION NO : -

NIC NO : 200577001310

Acknowledgements

I would like to express my heartfelt gratitude to those who have supported and guided me throughout the development of this Casio Cafe M-Commerce application.

First and foremost, I extend my heartfelt appreciation to my lecturer, Mr Anjana Dilhara, for his invaluable guidance and expertise throughout this project. His insights and constructive feedback have been instrumental in shaping the technical aspects and overall direction of this mobile commerce platform. I am also deeply grateful to all the supportive lecturers who provided their knowledge and assistance during my learning journey.

I would also like to acknowledge the management and staff of Casio Cafe PVT (LTD) for their cooperation in providing the necessary information and requirements that laid the foundation for this project. The journey of transforming a real-world business need into a fully functional M-Commerce application has been both challenging and rewarding, and I am proud of the progress made.

Finally, I thank everyone who contributed directly or indirectly to the successful completion of this project.

Abstract

This project presents the development of an Android mobile commerce application for Casio Cafe PVT (LTD), aimed at enhancing the mobile beverage ordering experience for Sri Lankan customers. The application provides a user-friendly interface for customers to browse the cafe's menu, customise their orders, and complete purchases conveniently through their mobile devices.

The primary objective of this project is to create a seamless and efficient ordering platform that addresses the growing demand for mobile commerce solutions in the food and beverage industry. The application integrates essential features such as user authentication with OTP verification, menu browsing and searching, customisable beverage orders, shopping cart functionality, secure payment processing, real-time order tracking with push notifications, and Google Maps integration for delivery and pickup location management. Additionally, the application includes a dedicated staff panel for cafe employees to manage menu items, process orders, and monitor sales statistics.

Through this project, I have explored various aspects of mobile application development, including UI/UX design with a coffee-themed aesthetic, backend integration, database management, real-time notifications, sensor integration (Camera and GPS), and security measures. The insights gained and the challenges overcome have contributed to my growth as a software engineer and deepened my understanding of the dynamic field of Android mobile applications and M-Commerce solutions.

Table of Contents

Acknowledgements **1**

Abstract **2**

Table of Contents **3**

1. Introduction 4

    1.1. Motivation 5

    1.2. Problem Statement 5

2. Background 5

3. Specifications 6

4. Design 8

    4.1. Architecture Used 8

4.2. UML Diagrams 9

    4.2.1. Usecase diagram 9

    4.3. UI high-fidelity wireframe diagrams 10

5. Implementation 16

5.1. Implementation Aspects 18

    5.1.1. UI Designing 18

6. Results and Evaluation 25

7. Future Work 26

8. Conclusions 26

9. References 27

List of Abbreviations.

API	-	Application Programming Interface
UX / UI	-	User Experience / User Interface
SDK	-	Software Development Kit
JSON	-	JavaScript Object Notation
IDE	-	Integrated Development Environment
HTTP	-	Hypertext Transfer Protocol

1. Introduction

“Technology should make life better and easier for people. Mobile devices and applications have the power to bring services directly into the hands of users, allowing them to communicate, shop, learn, and work more efficiently than ever before.”

— Tim Cook

1.1. Motivation

The rise of M-Commerce platforms has transformed how people order food and beverages. However, many cafes in Sri Lanka still rely on traditional counter-based ordering systems, leading to long wait times, order confusion, and limited customer engagement. The Casio Cafe M-Commerce application aims to bridge this gap by providing a dedicated digital platform for beverage ordering and cafe management, enabling customers to browse menus, customise their orders, and track their purchases conveniently from their mobile devices while empowering cafe staff with efficient order management tools.

1.2. Problem Statement

Traditional cafe ordering systems in Sri Lanka often lead to inefficiencies such as long queues during peak hours, order miscommunication, and limited customer engagement beyond the physical visit. Customers face difficulties browsing the full menu, customising their beverage preferences, and tracking their order status. Cafe staff struggle with manual order management, leading to errors and delays in service delivery. The absence of real-time notifications, digital payment options, and order tracking systems further complicates the customer experience. As noted by industry experts, **"In today's fast-paced world, customers expect convenience and speed in every transaction, and the food and beverage industry must adapt to meet these digital expectations." (Kotler & Keller, 2016)** The Casio Cafe M-Commerce application addresses these challenges by providing a comprehensive mobile platform that enhances both customer satisfaction and operational efficiency.

"Mobile technology has become the bridge between customer desire and instant gratification in the food service industry." (Laudon & Traver, 2020)

2. Background

The food and beverage industry in Sri Lanka has experienced significant growth in recent years, with an increasing number of cafes and coffee shops catering to diverse customer preferences. However, the adoption of Mobile Commerce (M-Commerce) applications in this sector remains limited, with most establishments still relying on traditional ordering methods. While global brands have successfully implemented mobile ordering platforms, local cafes in Sri Lanka lack dedicated m-commerce solutions that cater to their specific operational needs and customer expectations. The Casio Cafe M-Commerce application aims to leverage modern mobile technology to provide a simple, efficient, and user-friendly solution tailored for Sri Lankan cafe businesses and their customers, enhancing the overall beverage ordering experience through digital innovation.

3. Specifications

The Casio Cafe M-Commerce application ensures secure authentication through user registration, sign-in, and password reset with email verification for enhanced security.

Customers can efficiently browse and search for beverages by category, keywords, or filters, while cafe staff have access to menu management interfaces to add, edit, or remove beverage items with detailed descriptions, images, and pricing.

Customers can add beverages to their wishlist for quick access and conveniently add items directly from their wishlist to the cart.

Every user can update their personal information such as name, email, phone number, delivery address, and preferences. Users can also toggle between customer and cafe staff roles when granted appropriate access permissions.

Order management provides customers with real-time status updates through push notifications, tracking orders from "Processing" to "Preparing," "Ready for Pickup," and "Delivered." Cafe staff receive instant notifications for new orders and can update order statuses accordingly.

The application integrates Google Maps to display cafe locations, enable customers to select delivery or pickup addresses, and assist with navigation and location-based services.

Secure payment processing is facilitated through integrated payment gateway support, allowing customers to complete transactions via credit/debit cards, mobile wallets, or cash on delivery options.

The administrative panel, also known as the admin dashboard, consists of summarised analytics with charts and visual representations for sales data, order statistics, and revenue tracking. It includes comprehensive management options for users, menu items, and orders.

The cafe staff dashboard provides real-time insights into active orders, daily sales performance, and quick actions for menu management, enabling efficient operational oversight.

Finally, the Casio Cafe M-Commerce application is optimised for performance with seamless interactions, maintaining a visually appealing user interface with a warm coffee-themed aesthetic and delivering an exceptional user experience. The application is scalable due to its well-architected design, supporting future growth and feature enhancements.

4. Design

4.1. Architecture Used

The Casio Cafe M-Commerce application follows a Feature-Based Modular Architecture with MVC (Model-View-Controller) principles:

- **Model Layer:** Data management
    - Firebase Realtime Database
    - Firebase Storage
    - Java Beans as Domains (User, Beverage, Order, Cart, Wishlist)

- **View Layer:** Activities and Fragments
    - Activities (Home, Menu, Cart, Checkout, Order Tracking, Staff Dashboard, Admin Panel)
    - Fragments (Beverage Details, User Profile, Order History)
    - Resource layouts
    - Component cards (Beverage cards, Order cards)

- **Controller Layer:** Adapter and Helper classes handling business logic
    - Obtain device permissions (Camera, GPS, Notifications)
    - Email verification and notifications
    - Real-time push notifications for order updates
    - Cart & Wishlist management
    - Order status management
    - Input validations (Email, Password, Phone Number)
    - Payment processing integration
    - Google Maps integration for location services
    - Image upload and storage management

4.2. UML Diagrams

4.2.1. Usecase diagram

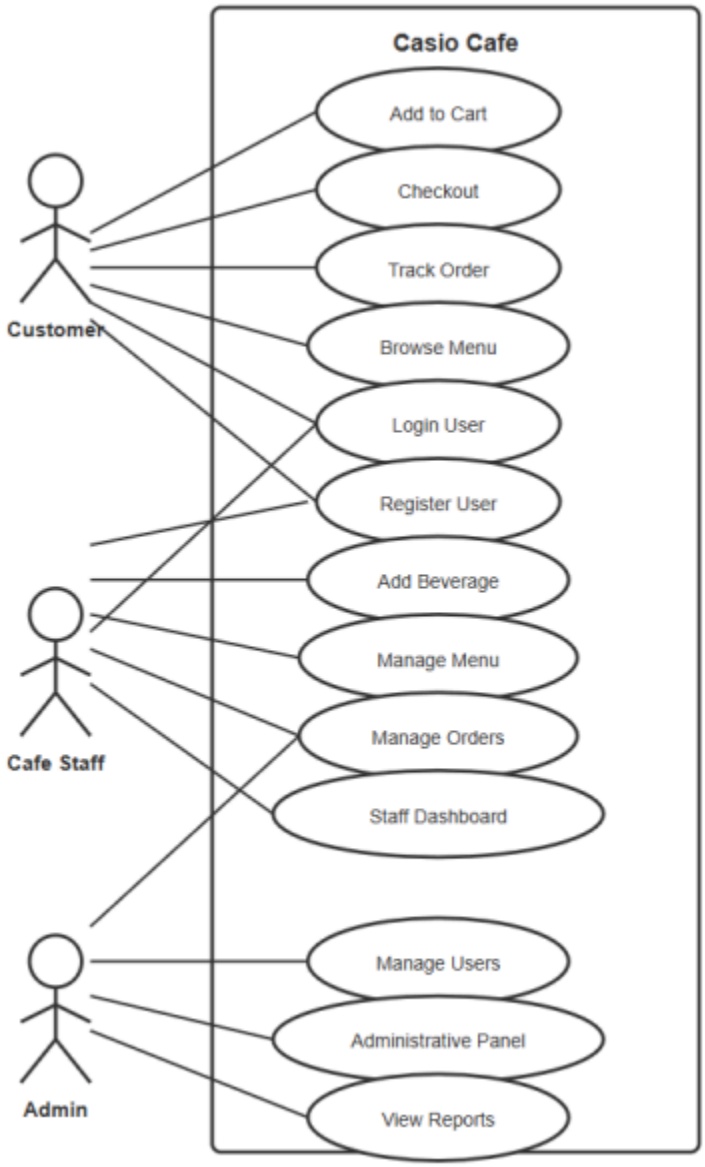


Figure 1: Use Case Diagram – drawn with [Draw.io]

4.3. UI high-fidelity wireframe diagrams



Figure 2: Splash Screen

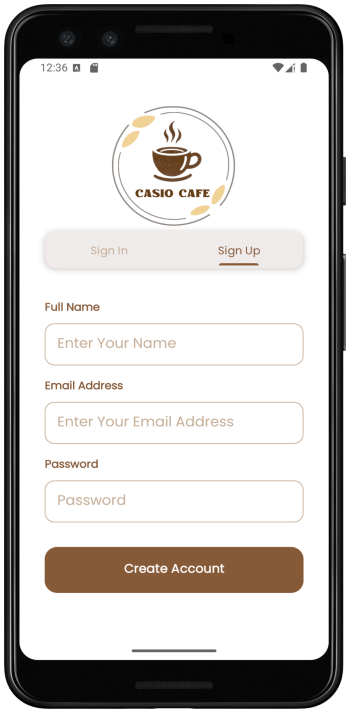


Figure 3: User Registration

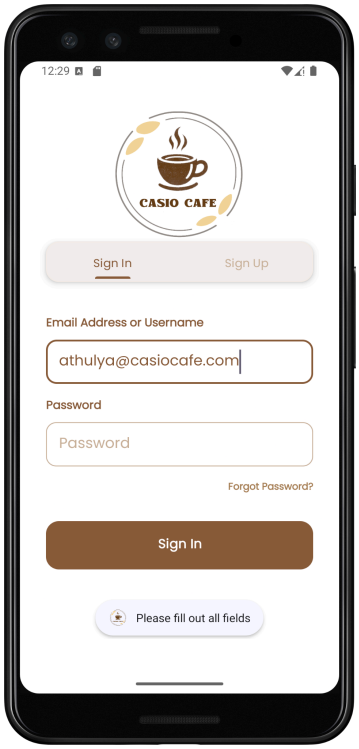


Figure 4: User Sign-In

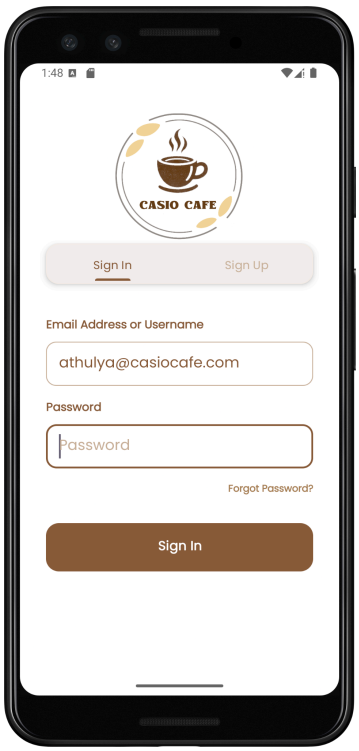


Figure 5: Forgot Password

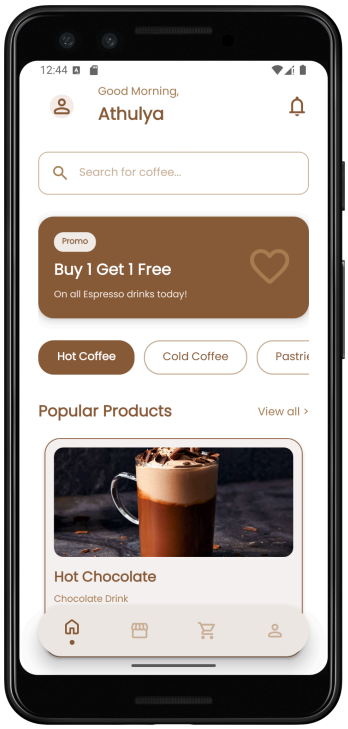


Figure 6: Home page

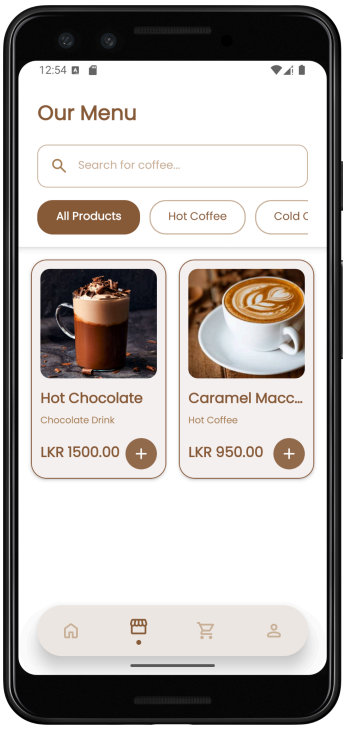


Figure 7: Menu Page

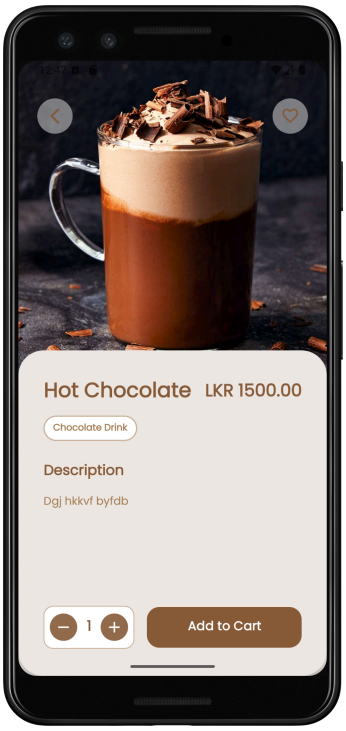


Figure 8: single product view

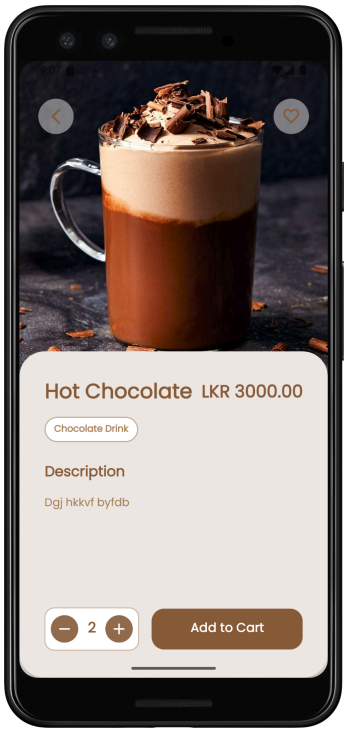


Figure 9: add more items

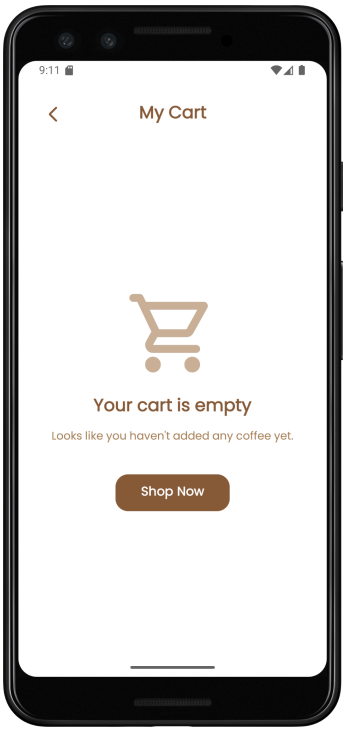


Figure 10: User Cart

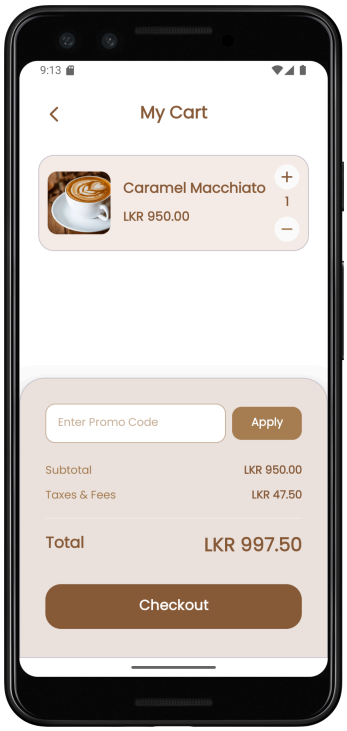


Figure 11: Add item to cart

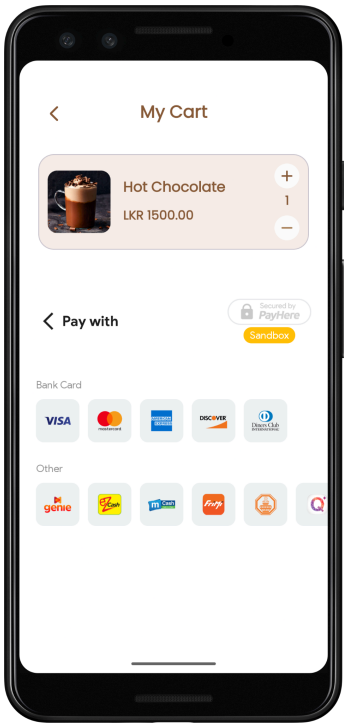


Figure 12: Payment Process

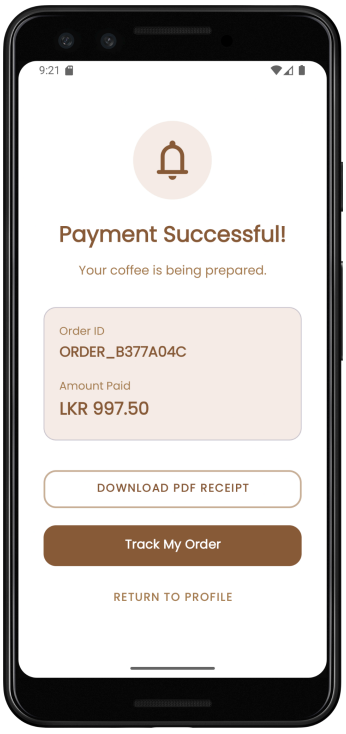


Figure 13: Payment Success

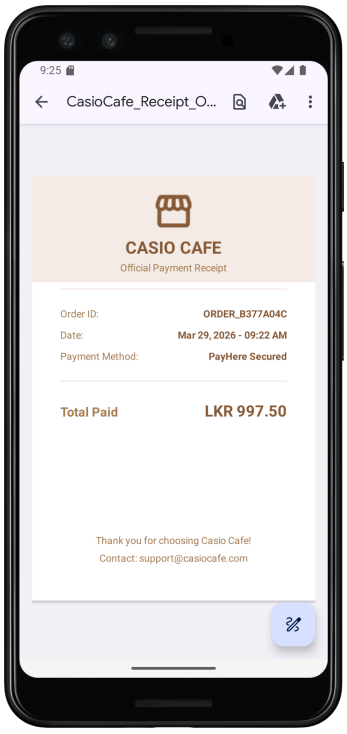


Figure 14: Downloaded Receipt

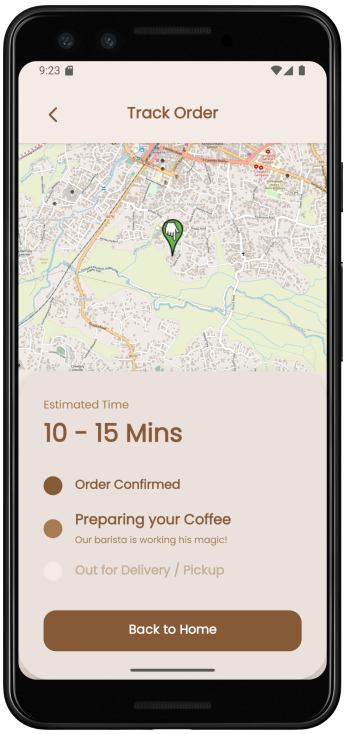


Figure 15: Track your Order

s

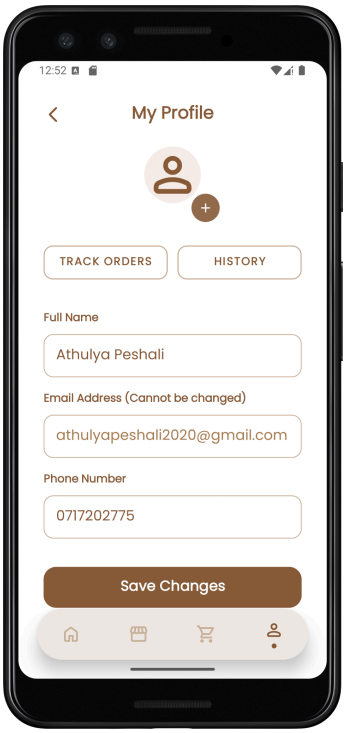


Figure 16: User Profile

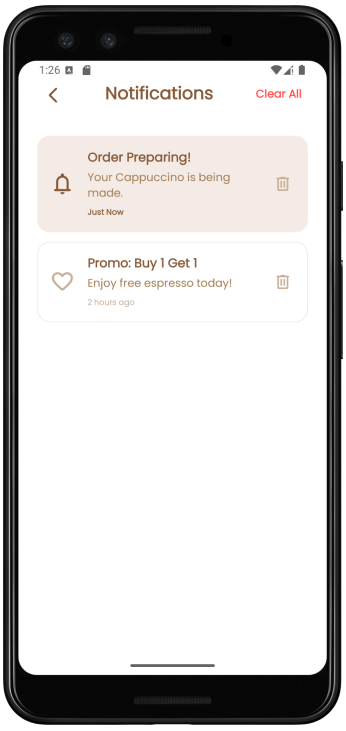


Figure 17 User Notification

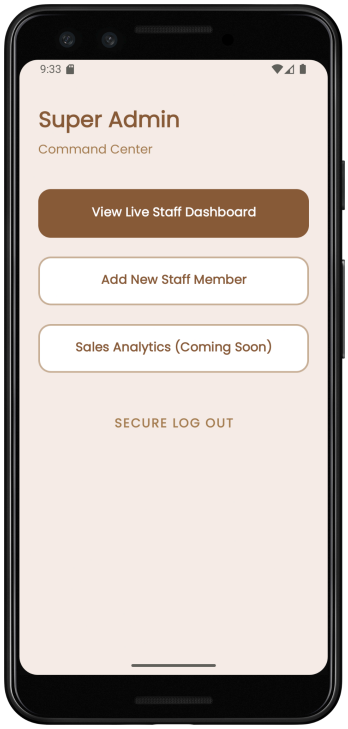


Figure 18: Admin Dashboard

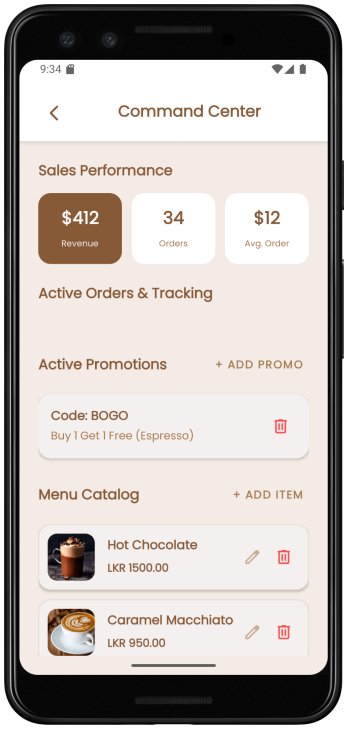


Figure 19 Active Orders

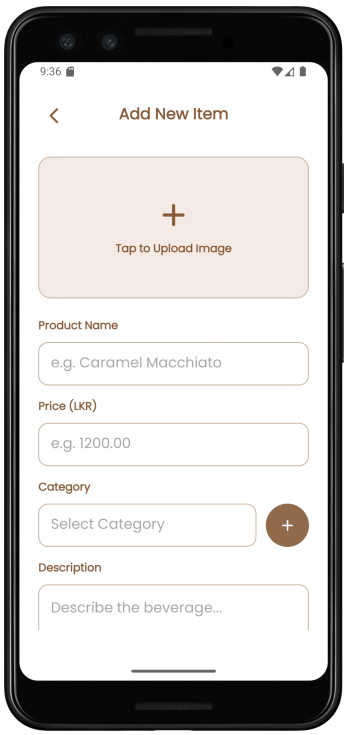


Figure 20: Add a New Item

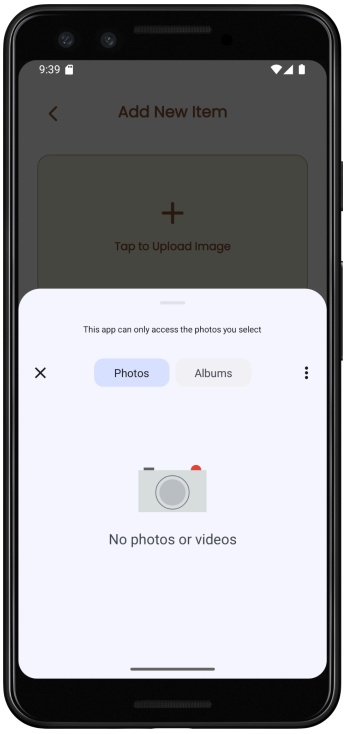


Figure 21: Image Upload

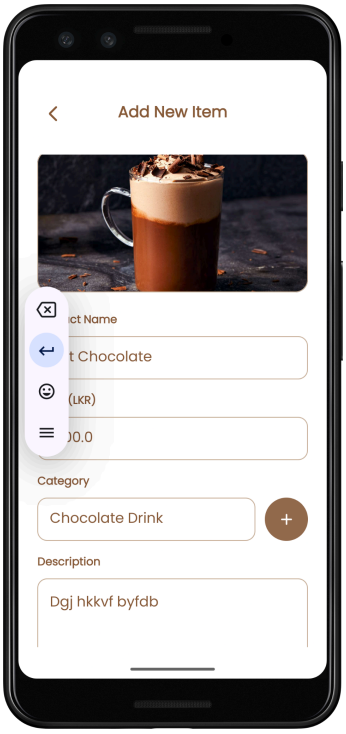


Figure 22: Edit Item

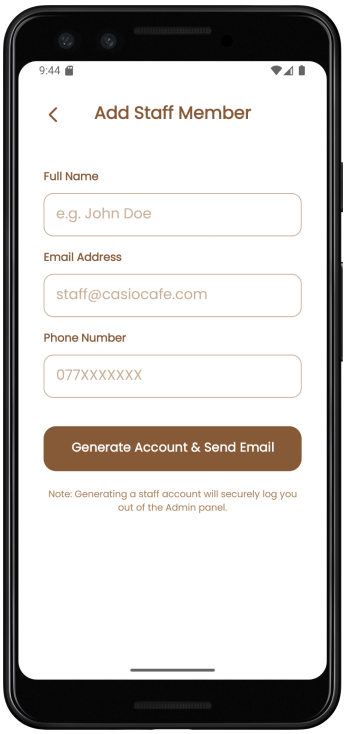


Figure 23: Add a staff Member



Figure 24: Staff Profile

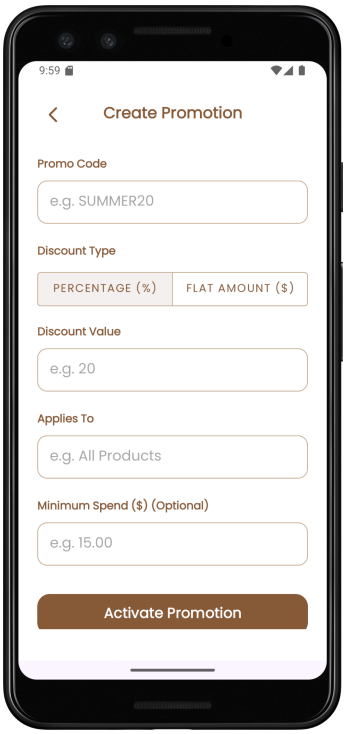


Figure 25: Add Promotions

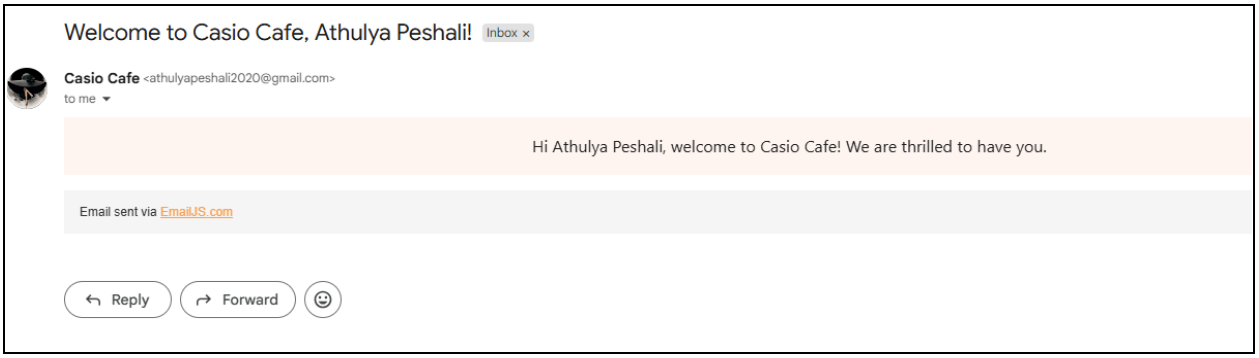


Figure 26: Email sending for new customer

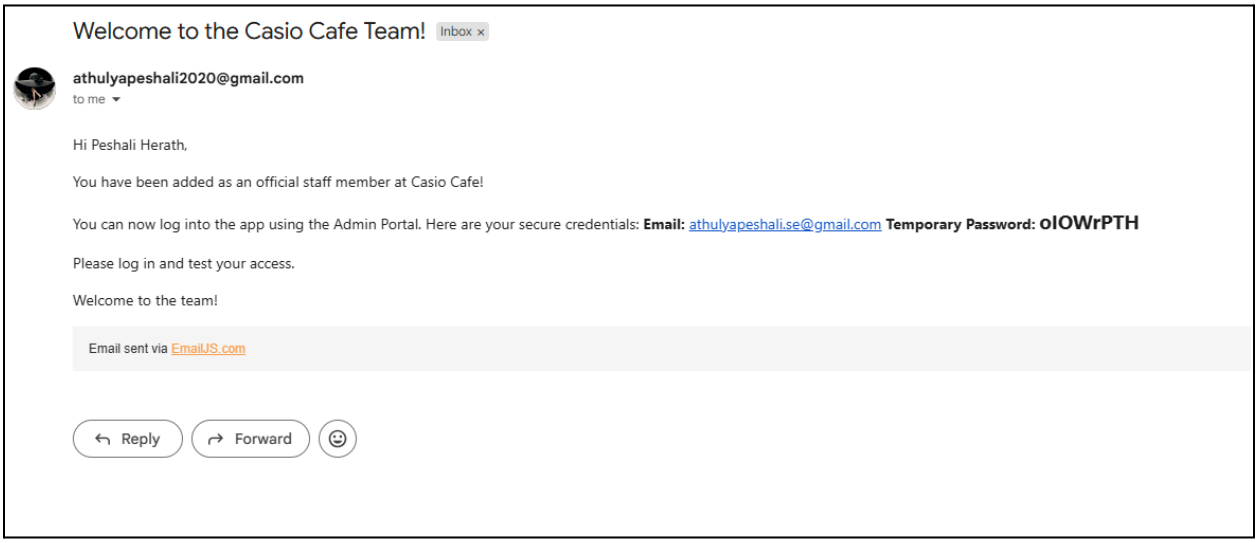


Figure 27: Temporary password sending for new singing cafe staff member



Figure 28: Password reset email sending

5. Implementation

Programming Language:

- Java

Database:

- Firebase Realtime Database (as the main database)
- SharedPreferences (as local data storage)
- SQLite (for offline caching and data persistence)

Storage:

- Firebase Storage Bucket (for beverage images and user profile pictures)

Third-Party Libraries:

- Glide (image loading and caching)
- Firebase UI Auth (authentication)
- Google Maps API (location services and navigation)
- Payment Gateway SDK (for secure transactions)
- Volley (for network requests and API communication)
- Picasso (alternative image loading)

User Roles:

- **Customer:**
    - Browse menu
    - Search and filter beverages
    - Add to cart and wishlist
    - Checkout and make payments
    - Track orders in real-time
    - Manage user profile

- **Cafe Staff:**
    - Add/edit/remove beverages
    - Manage menu items
    - Receive and process orders
    - Update order status
    - View staff dashboard and sales statistics
- **Admin:**
    - Manage users (customers and staff)
    - Manage menu items
    - Manage orders
    - View sales reports and analytics
    - Access administrative panel with data visualization

5.1. Implementation Aspects

1. Role-Based Routing (Security & UX)

```
import android.content.Intent;
import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;

import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.auth.FirebaseUser;
import com.google.firebase.firestore.FirebaseFirestore;

public class MainActivity extends AppCompatActivity {
```

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    // 1. Switch out of the Splash theme to your normal theme
    setTheme(R.style.Theme_CasioCafe);
    super.onCreate(savedInstanceState);

    // 2. Check if the user is currently logged in
    FirebaseAuth mAuth = FirebaseAuth.getInstance();
    FirebaseUser currentUser = mAuth.getCurrentUser();

    if (currentUser != null) {
        // User IS logged in -> Check their role to route them safely!
        routeUserSafely(currentUser.getUid());
    } else {
        // User is NOT logged in -> Send to Sign In / Sign Up Screen
        startActivity(new Intent( packageContext: MainActivity.this, AuthActivity.class));
        finish();
    }
}
```

```
    1 usage
    private void routeUserSafely(String uid) {
        FirebaseFirestore.getInstance().collection( collectionPath: "Users").document(uid).get()
                .addOnSuccessListener( DocumentSnapshot documentSnapshot -> {
                    String role = documentSnapshot.getString( field: "role");

                    Intent intent;
                    if ("admin".equals(role)) {
                        intent = new Intent( packageContext: MainActivity.this, SuperAdminDashboardActivity.class);
                    } else if ("staff".equals(role)) {
                        intent = new Intent( packageContext: MainActivity.this, StaffDashboardActivity.class);
                    } else {
                        // Standard Customer
                        intent = new Intent( packageContext: MainActivity.this, HomeActivity.class);
                    }

                    startActivity(intent);

                    // Close the launcher so the user can't hit "back" to return to the splash screen
                    finish();
                })
                .addOnFailureListener( Exception e -> {
                    // Safety Fallback: If the database fails (e.g., no internet connection),
                    // force them to log in again safely rather than sending them to the wrong screen.
                    FirebaseAuth.getInstance().signOut();
                    startActivity(new Intent( packageContext: MainActivity.this, AuthActivity.class));
                    finish();
                });
    }
}
```

2. Real-Time Menu Management

```java
package com.athulya.casiocafe;

import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import android.widget.TextView;
import android.widget.Toast;

import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.firestore.FirebaseFirestore;
import com.google.firebase.firestore.QueryDocumentSnapshot;

import java.util.ArrayList;

public class HomeActivity extends AppCompatActivity {
```

```java
public class HomeActivity extends AppCompatActivity {

    3 usages
    private RecyclerView rvPopularProducts;
    3 usages
    private ProductAdapter productAdapter;
    4 usages
    private ArrayList<Product> productList;
    3 usages
    private FirebaseFirestore db;
    3 usages
    private FirebaseAuth mAuth; // ADDED THIS

    4 usages
    private TextView tvUserName; // ADDED THIS
```

```java
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    EdgeToEdge.enable( $this$enableEdgeToEdge: this);
    setContentView(R.layout.activity_home);

    View rootView = findViewById(R.id.main_home);
    ViewCompat.setOnApplyWindowInsetsListener(rootView, ( View v,  WindowInsetsCompat insets) -> {
        Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
        v.setPadding(systemBars.left, systemBars.top, systemBars.right,  bottom: 0);
        return insets;
    });

    // Initialize Firebase
    db = FirebaseFirestore.getInstance();
    mAuth = FirebaseAuth.getInstance();

    // Bind the Username TextView and load the name!
    tvUserName = findViewById(R.id.tv_home_username);
    loadUserName();

    // Initialize RecyclerView
    rvPopularProducts = findViewById(R.id.rv_popular_products);
    rvPopularProducts.setLayoutManager(new LinearLayoutManager( context: this, LinearLayoutManager.HORIZONTAL,  reverseLayout: false));

    productList = new ArrayList<>();
    productAdapter = new ProductAdapter( context: this, productList);
    rvPopularProducts.setAdapter(productAdapter);

    // Fetch Data
    loadPopularProducts();
```

```
    // Fetch Data
    loadPopularProducts();

    // Nav Bar Routing
    findViewById(R.id.nav_shop).setOnClickListener( View v -> {
        startActivity(new Intent( packageContext: HomeActivity.this, ShopActivity.class));
        overridePendingTransition( enterAnim: 0,  exitAnim: 0);
        finish();
    });

    findViewById(R.id.nav_cart).setOnClickListener( View v -> {
        startActivity(new Intent( packageContext: HomeActivity.this, CartActivity.class));
        overridePendingTransition( enterAnim: 0,  exitAnim: 0);
    });

    findViewById(R.id.nav_profile).setOnClickListener( View v -> {
        startActivity(new Intent( packageContext: HomeActivity.this, ProfileActivity.class));
        overridePendingTransition( enterAnim: 0,  exitAnim: 0);
        finish();
    });

    findViewById(R.id.btn_notifications).setOnClickListener( View v -> {
        startActivity(new Intent( packageContext: HomeActivity.this, NotificationsActivity.class));
    });
}
```

```
    // NEW METHOD TO FETCH USERNAME
    1 usage
    private void loadUserName() {
        if (mAuth.getCurrentUser() != null) {
            String userId = mAuth.getCurrentUser().getUid();

            db.collection( collectionPath: "Users").document(userId).get()
                    .addOnSuccessListener( DocumentSnapshot documentSnapshot -> {
                        if (documentSnapshot.exists()) {
                            String fullName = documentSnapshot.getString( field: "fullName");
                            if (fullName != null && !fullName.isEmpty()) {
                                // Split the string at the first space to just get the First Name
                                String firstName = fullName.split( regex: " ")[0];
                                tvUserName.setText(firstName);
                            } else {
                                tvUserName.setText("Guest");
                            }
                        }
                    })
                    .addOnFailureListener( Exception e -> tvUserName.setText("Welcome"));
        }
    }
```

```
    1 usage
    private void loadPopularProducts() {
        db.collection( collectionPath: "Products").get()
                .addOnSuccessListener( QuerySnapshot queryDocumentSnapshots -> {
                    productList.clear();
                    for (QueryDocumentSnapshot document : queryDocumentSnapshots) {
                        Product product = document.toObject(Product.class);
                        productList.add(product);
                    }
                    productAdapter.notifyDataSetChanged();
                })
                .addOnFailureListener( Exception e -> {
                    Toast.makeText( context: this, text: "Failed to load popular items", Toast.LENGTH_SHORT).show();
                });
    }
}
```

3. Payment Gateway Integration

```
package com.athulya.casiocafe;

import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import android.widget.EditText;
import android.widget.LinearLayout;
import android.widget.TextView;
import android.widget.Toast;

import androidx.activity.EdgeToEdge;
import androidx.activity.result.ActivityResultLauncher;
import androidx.activity.result.contract.ActivityResultContracts;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

import com.google.android.material.card.MaterialCardView;
import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.firestore.FirebaseFirestore;

import java.math.BigInteger;
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;
import java.text.DecimalFormat;
import java.util.UUID;

import lk.payhere.androidsdk.PHConfigs;
import lk.payhere.androidsdk.PHConstants;
import lk.payhere.androidsdk.PHMainActivity;
import lk.payhere.androidsdk.PHResponse;
import lk.payhere.androidsdk.model.InitRequest;
import lk.payhere.androidsdk.model.StatusResponse;

public class CartActivity extends AppCompatActivity {
```

```
private final String MERCHANT_ID = "1222469";
1 usage
private final String MERCHANT_SECRET = "MzE0OTA2OTE5NDE1Njg2OTcxOTAxNTAyNTY1MjIxMjc5MjgwODY4Mw==";

5 usages
private RecyclerView rvCartItems;
2 usages
private CartAdapter cartAdapter;
2 usages
private TextView tvSubtotal, tvTaxes, tvTotal;
3 usages
private LinearLayout layoutEmptyCart;
3 usages
private MaterialCardView checkoutPanel;

1 usage
private double taxRate = 0.05;
2 usages
private double discount = 0.0;
6 usages
private double currentFinalTotal = 0.0;

3 usages
private String currentOrderId = "";
```

```
// Firebase & User Data Variables
4 usages
private FirebaseAuth mAuth;
2 usages
private FirebaseFirestore db;
2 usages
private String customerFirstName = "Guest";
3 usages
private String customerLastName = "User";
2 usages
private String customerEmail = "no-email@example.com";
2 usages
private String customerPhone = "0000000000";
```

```
// This Launcher listens for the result from the PayHere popup
1 usage
private final ActivityResultLauncher<Intent> payHereLauncher = registerForActivityResult(
        new ActivityResultContracts.StartActivityForResult(),
        ActivityResult result -> {
            if (result.getData() != null && result.getData().hasExtra(PHConstants.INTENT_EXTRA_RESULT)) {
                PHResponse<StatusResponse>
                        response = (PHResponse<StatusResponse>) result.getData().getSerializableExtra(PHConstants.INTENT_EXTRA_RESULT);

                if (response != null && response.isSuccess()) {
                    Toast.makeText( context: this,  text: "Payment Successful!", Toast.LENGTH_LONG).show();
                    CartManager.getInstance().clearCart();

                    // NEW ROUTING LOGIC:
                    Intent intent = new Intent( packageContext: CartActivity.this, PaymentSuccessActivity.class);
                    intent.putExtra( name: "ORDER_ID", currentOrderId);
                    intent.putExtra( name: "TOTAL_AMOUNT", currentFinalTotal);
                    startActivity(intent);
                    finish();
                } else {
                    Toast.makeText( context: this,  text: "Payment Failed: " + (response != null ? response.toString() : "Unknown Error"),
                            Toast.LENGTH_LONG).show();
                }
            } else {
                Toast.makeText( context: this,  text: "Payment Canceled", Toast.LENGTH_SHORT).show();
            }
        }
);
```

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    EdgeToEdge.enable( $this$enableEdgeToEdge: this);
    setContentView(R.layout.activity_cart);

    View rootView = findViewById(R.id.main_cart);
    ViewCompat.setOnApplyWindowInsetsListener(rootView, ( View v,  WindowInsetsCompat insets) -> {
        Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
        v.setPadding( left: 0,  top: 0,  right: 0, systemBars.bottom);
        return insets;
    });

    // Initialize Firebase
    mAuth = FirebaseAuth.getInstance();
    db = FirebaseFirestore.getInstance();

    tvSubtotal = findViewById(R.id.tv_subtotal);
    tvTaxes = findViewById(R.id.tv_taxes);
    tvTotal = findViewById(R.id.tv_total);
    layoutEmptyCart = findViewById(R.id.layout_empty_cart);
    checkoutPanel = findViewById(R.id.checkout_panel);

    findViewById(R.id.btn_cart_back).setOnClickListener( View v -> finish());

    // Setup RecyclerView
    rvCartItems = findViewById(R.id.rv_cart_items);
    rvCartItems.setLayoutManager(new LinearLayoutManager( context: this));
    cartAdapter = new CartAdapter( context: this, CartManager.getInstance().getCartItems(), this::calculateTotals);
    rvCartItems.setAdapter(cartAdapter);

    calculateTotals();
    fetchUserDetails(); // Fetch the logged-in user's data in the background
```

```
    // Promo Logic
    findViewById(R.id.btn_apply_promo).setOnClickListener( View v -> {
        EditText etPromo = findViewById(R.id.et_promo_code);
        String code = etPromo.getText().toString().trim().toUpperCase();

        if (code.equals("FREECOFFEE") || code.equals("BOGO")) {
            discount = 200.00;
            Toast.makeText( context: this,  text: "Promo Applied! LKR 200 off.", Toast.LENGTH_SHORT).show();
            calculateTotals();
        } else {
            Toast.makeText( context: this,  text: "Invalid Promo Code", Toast.LENGTH_SHORT).show();
        }
    });

    // Checkout Logic -> Trigger PayHere
    findViewById(R.id.btn_checkout).setOnClickListener( View v -> {
        if (CartManager.getInstance().getCartItems().isEmpty()) return;
        startPayHereCheckout();
    });

    findViewById(R.id.btn_shop_now).setOnClickListener( View v -> {
        startActivity(new Intent( packageContext: CartActivity.this, ShopActivity.class));
        finish();
    });
}
```

```
1 usage
private void fetchUserDetails() {
    if (mAuth.getCurrentUser() != null) {
        customerEmail = mAuth.getCurrentUser().getEmail(); // Default to Auth email
        String userId = mAuth.getCurrentUser().getUid();

        db.collection( collectionPath: "Users").document(userId).get()
                .addOnSuccessListener( DocumentSnapshot documentSnapshot -> {
                    if (documentSnapshot.exists()) {
                        String fullName = documentSnapshot.getString( field: "fullName");
                        String phone = documentSnapshot.getString( field: "phone");

                        // PayHere needs First and Last name separately, so we split the full name
                        if (fullName != null && !fullName.trim().isEmpty()) {
                            String[] nameParts = fullName.trim().split( regex: " ",  limit: 2);
                            customerFirstName = nameParts[0];
                            if (nameParts.length > 1) {
                                customerLastName = nameParts[1];
                            } else {
                                customerLastName = "";
                            }
                        }
                        if (phone != null && !phone.isEmpty()) {
                            customerPhone = phone;
                        }
                    }
                })
                .addOnFailureListener( Exception e -> {
                    // If it fails, we just fall back to the default "Guest" values
                });
    }
}
```

```
3 usages
private void calculateTotals() {
    if (CartManager.getInstance().getCartItems().isEmpty()) {
        rvCartItems.setVisibility(View.GONE);
        checkoutPanel.setVisibility(View.GONE);
        layoutEmptyCart.setVisibility(View.VISIBLE);
        return;
    } else {
        rvCartItems.setVisibility(View.VISIBLE);
        checkoutPanel.setVisibility(View.VISIBLE);
        layoutEmptyCart.setVisibility(View.GONE);
    }

    double subtotal = CartManager.getInstance().getSubtotal();
    double taxes = subtotal * taxRate;
    currentFinalTotal = (subtotal + taxes) - discount;

    if (currentFinalTotal < 0) currentFinalTotal = 0;

    tvSubtotal.setText(String.format("LKR %.2f", subtotal));
    tvTaxes.setText(String.format("LKR %.2f", taxes));
    tvTotal.setText(String.format("LKR %.2f", currentFinalTotal));
}
```

```
1 usage
private void startPayHereCheckout() {
    String orderId = "ORDER_" + UUID.randomUUID().toString().substring(0, 8).toUpperCase();
    String currency = "LKR";
    currentOrderId = "ORDER_" + UUID.randomUUID().toString().substring(0, 8).toUpperCase();

    InitRequest req = new InitRequest();
    req.setMerchantId(MERCHANT_ID);
    req.setMerchantSecret(MERCHANT_SECRET); // <-- NEW: Just pass the secret directly!
    req.setCurrency(currency);
    req.setAmount(currentFinalTotal);
    req.setOrderId(orderId);
    req.setItemsDescription("Casio Cafe App Order");
    req.setCustom1("In-App Purchase");
    req.setCustom2("");
    req.setOrderId(currentOrderId);

    // Pass the Dynamic Firebase Data to PayHere
    req.getCustomer().setFirstName(customerFirstName);
    req.getCustomer().setLastName(customerLastName);
    req.getCustomer().setEmail(customerEmail);
    req.getCustomer().setPhone(customerPhone);

    req.getCustomer().getAddress().setAddress("No.1, Galle Road");
    req.getCustomer().getAddress().setCity("Colombo");
    req.getCustomer().getAddress().setCountry("Sri Lanka");

    // (We completely removed the manual hash generation code here)

    Intent intent = new Intent( packageContext: this, PHMainActivity.class);
    intent.putExtra(PHConstants.INTENT_EXTRA_DATA, req);
    PHConfigs.setBaseUrl(PHConfigs.SANDBOX_URL);
    payHereLauncher.launch(intent);
}

}
```

4. Shopping Cart Logic (State Management)

```java
package com.athulya.casiocafe;

import java.util.ArrayList;

11 usages
public class CartManager {
    3 usages
    private static CartManager instance;
    7 usages
    private ArrayList<CartItem> cartItems;

    1 usage
    private CartManager() { cartItems = new ArrayList<>(); }

    public static CartManager getInstance() {
        if (instance == null) {
            instance = new CartManager();
        }
        return instance;
    }

    3 usages
    public ArrayList<CartItem> getCartItems() { return cartItems; }

    2 usages
    public void addItem(Product product, int quantity) {
        // Check if the item already exists in the cart. If so, just increase the quantity.
        for (CartItem item : cartItems) {
            if (item.getProduct().getId().equals(product.getId())) {
                item.setQuantity(item.getQuantity() + quantity);
                return;
            }
        }
        // Otherwise, add it as a brand new row
        cartItems.add(new CartItem(product, quantity));
    }
```

```
    1 usage
    public void removeItem(CartItem item) { cartItems.remove(item); }

    1 usage
    public double getSubtotal() {
        double total = 0;
        for (CartItem item : cartItems) {
            total += item.getTotalPrice();
        }
        return total;
    }

    1 usage
    public void clearCart() { cartItems.clear(); }
}
```

5. Product Management & Image Upload

```java
package com.athulya.casiocafe;

import android.content.Intent;
import android.net.Uri;
import android.os.Bundle;
import android.util.Base64;
import android.view.View;
import android.widget.ArrayAdapter;
import android.widget.AutoCompleteTextView;
import android.widget.EditText;
import android.widget.ImageView;
import android.widget.Toast;

import androidx.activity.EdgeToEdge;
import androidx.activity.result.ActivityResultLauncher;
import androidx.activity.result.contract.ActivityResultContracts;
import androidx.appcompat.app.AlertDialog;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

import com.google.android.material.button.MaterialButton;
import com.google.firebase.firestore.FirebaseFirestore;

import org.json.JSONObject;

import java.io.BufferedReader;
import java.io.ByteArrayOutputStream;
import java.io.InputStream;
import java.io.InputStreamReader;
import java.io.OutputStreamWriter;
import java.net.HttpURLConnection;
import java.net.URL;
import java.net.URLEncoder;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.UUID;
import java.util.concurrent.Executors;

public class AddProductActivity extends AppCompatActivity {
```

```
2 usages
private ArrayList<String> categoryList = new ArrayList<>(Arrays.asList("Hot Coffee", "Cold Coffee", "Pastries"));
3 usages
private ArrayAdapter<String> adapter;
2 usages
private FirebaseFirestore db;

4 usages
private Uri imageUri;
3 usages
private ImageView ivPreviewImage;
8 usages
private MaterialButton btnSaveProduct;

// Launcher to pick image from gallery
1 usage
private final ActivityResultLauncher<Intent> pickImageLauncher = registerForActivityResult(
        new ActivityResultContracts.StartActivityForResult(),
        ActivityResult result -> {
            if (result.getResultCode() == RESULT_OK && result.getData() != null) {
                imageUri = result.getData().getData();
                ivPreviewImage.setImageURI(imageUri);
                ivPreviewImage.setVisibility(View.VISIBLE);
            }
        }
);
```

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    EdgeToEdge.enable( $this$enableEdgeToEdge: this);
    setContentView(R.layout.activity_add_product);

    db = FirebaseFirestore.getInstance();
    ivPreviewImage = findViewById(R.id.iv_preview_image);
    btnSaveProduct = findViewById(R.id.btn_save_product);

    View rootView = findViewById(R.id.main_add_product);
    ViewCompat.setOnApplyWindowInsetsListener(rootView, ( View v, WindowInsetsCompat insets) -> {
        Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
        v.setPadding( left: 0,  top: 0,  right: 0, systemBars.bottom);
        return insets;
    });

    AutoCompleteTextView dropdownCategory = findViewById(R.id.dropdown_category);
    adapter = new ArrayAdapter<>( context: this, android.R.layout.simple_dropdown_item_1line, categoryList);
    dropdownCategory.setAdapter(adapter);
    dropdownCategory.setOnClickListener( View v -> dropdownCategory.showDropDown());

    findViewById(R.id.btn_add_category).setOnClickListener( View v -> showAddCategoryDialog(dropdownCategory));
    findViewById(R.id.btn_back_add).setOnClickListener( View v -> finish());

    // Open Gallery
    findViewById(R.id.cv_upload_image).setOnClickListener( View v -> {
        Intent intent = new Intent();
        intent.setType("image/*");
        intent.setAction(Intent.ACTION_GET_CONTENT);
        pickImageLauncher.launch(Intent.createChooser(intent,  title: "Select Picture"));
    });

    // Save Button
    btnSaveProduct.setOnClickListener( View v -> uploadData(dropdownCategory));
}
```

```java
private void uploadData(AutoCompleteTextView dropdownCategory) {
    EditText etName = findViewById(R.id.et_product_name);
    EditText etPrice = findViewById(R.id.et_product_price);
    EditText etDesc = findViewById(R.id.et_product_desc);

    String name = etName.getText().toString().trim();
    String priceStr = etPrice.getText().toString().trim();
    String category = dropdownCategory.getText().toString().trim();
    String desc = etDesc.getText().toString().trim();

    if (name.isEmpty() || priceStr.isEmpty() || category.isEmpty()) {
        Toast.makeText( context: this,  text: "Please fill in all required fields", Toast.LENGTH_SHORT).show();
        return;
    }

    double price = Double.parseDouble(priceStr);
    String uniqueID = UUID.randomUUID().toString();

    btnSaveProduct.setEnabled(false);
    btnSaveProduct.setText("Uploading...");

    if (imageUri != null) {
        uploadImageToImgBBAndSave(uniqueID, name, price, category, desc);
    } else {
        saveProductToFirestore(uniqueID, name, price, category, desc,  imageUrl: "");
    }
}
```

```java
private void uploadImageToImgBBAndSave(String id, String name, double price, String category, String desc) {
    // Run network task in background to prevent app freeze
    Executors.newSingleThreadExecutor().execute(() -> {
        try {
            // 1. Convert Image to Base64
            InputStream imageStream = getContentResolver().openInputStream(imageUri);
            ByteArrayOutputStream baos = new ByteArrayOutputStream();
            byte[] buffer = new byte[1024];
            int bytesRead;
            while ((bytesRead = imageStream.read(buffer)) != -1) {
                baos.write(buffer,  off: 0, bytesRead);
            }
            String base64Image = Base64.encodeToString(baos.toByteArray(), Base64.DEFAULT);

            // 2. Setup HTTP POST to ImgBB
            String API_KEY = "dd88074a869feca844dbf925be096d1e"; // <--- PUT YOUR KEY HERE
            URL url = new URL( spec: "https://api.imgbb.com/1/upload?key=" + API_KEY);
            HttpURLConnection conn = (HttpURLConnection) url.openConnection();
            conn.setRequestMethod("POST");
            conn.setDoOutput(true);

            // 3. Send Data
            String data = URLEncoder.encode( s: "image",  enc: "UTF-8") + "=" + URLEncoder.encode(base64Image,  enc: "UTF-8");
            OutputStreamWriter wr = new OutputStreamWriter(conn.getOutputStream());
            wr.write(data);
            wr.flush();

            // 4. Read Response
            BufferedReader reader = new BufferedReader(new InputStreamReader(conn.getInputStream()));
            StringBuilder response = new StringBuilder();
            String line;
            while ((line = reader.readLine()) != null) {
                response.append(line);
            }
```

```
// 2. Setup HTTP POST to ImgBB
String API_KEY = "dd88074a869feca844dbf925be096d1e"; // <--- PUT YOUR KEY HERE
URL url = new URL( spec: "https://api.imgbb.com/1/upload?key=" + API_KEY);
HttpURLConnection conn = (HttpURLConnection) url.openConnection();
conn.setRequestMethod("POST");
conn.setDoOutput(true);

// 3. Send Data
String data = URLEncoder.encode( s: "image",  enc: "UTF-8") + "=" + URLEncoder.encode(base64Image,  enc: "UTF-8");
OutputStreamWriter wr = new OutputStreamWriter(conn.getOutputStream());
wr.write(data);
wr.flush();

// 4. Read Response
BufferedReader reader = new BufferedReader(new InputStreamReader(conn.getInputStream()));
StringBuilder response = new StringBuilder();
String line;
while ((line = reader.readLine()) != null) {
    response.append(line);
}

// 5. Extract URL from JSON
JSONObject jsonObject = new JSONObject(response.toString());
String imageUrl = jsonObject.getJSONObject( name: "data").getString( name: "url");

// 6. Go back to Main Thread to save to Firestore
runOnUiThread(() -> saveProductToFirestore(id, name, price, category, desc, imageUrl));
```

```
        } catch (Exception e) {
            e.printStackTrace();
            runOnUiThread(() -> {
                Toast.makeText( context: AddProductActivity.this,  text: "Image Upload Failed!", Toast.LENGTH_SHORT).show();
                btnSaveProduct.setEnabled(true);
                btnSaveProduct.setText("Publish to Menu");
            });
        }
    });
}

2 usages
private void saveProductToFirestore(String id, String name, double price, String category, String desc, String imageUrl) {
    Product newProduct = new Product(id, name, price, category, desc, imageUrl);

    db.collection( collectionPath: "Products").document(id).set(newProduct)
            .addOnSuccessListener( Void aVoid -> {
                Toast.makeText( context: this,  text: "Product Published Successfully!", Toast.LENGTH_SHORT).show();
                finish();
            })
            .addOnFailureListener( Exception e -> {
                Toast.makeText( context: this,  text: "Error saving: " + e.getMessage(), Toast.LENGTH_LONG).show();
                btnSaveProduct.setEnabled(true);
                btnSaveProduct.setText("Publish to Menu");
            });
}
```

```
1 usage
private void showAddCategoryDialog(AutoCompleteTextView dropdown) {
    AlertDialog.Builder builder = new AlertDialog.Builder( context: this);
    builder.setTitle("Add New Category");
    final EditText input = new EditText( context: this);
    input.setHint("e.g. Seasonal Drinks");
    builder.setView(input);
    builder.setPositiveButton( text: "Add", ( DialogInterface dialog, int which) -> {
        String newCategory = input.getText().toString().trim();
        if (!newCategory.isEmpty()) {
            categoryList.add(newCategory);
            adapter.notifyDataSetChanged();
            dropdown.setText(newCategory, filter: false);
        }
    });
    builder.setNegativeButton( text: "Cancel", ( DialogInterface dialog, int which) -> dialog.cancel());
    builder.show();
}
}
```

6. Firebase Authentication Logic

```
package com.athulya.casiocafe;

import android.os.Bundle;
import android.view.View;
import android.widget.EditText;
import android.widget.Toast;

import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

import com.google.android.material.button.MaterialButton;
import com.google.firebase.auth.FirebaseAuth;

public class ForgotPasswordActivity extends AppCompatActivity {
```

```
    private FirebaseAuth mAuth;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable( $this$enableEdgeToEdge: this);
        setContentView(R.layout.activity_forgot_password);

        View rootView = findViewById(R.id.main_forgot_password);
        ViewCompat.setOnApplyWindowInsetsListener(rootView, ( View v, WindowInsetsCompat insets) -> {
            Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
            v.setPadding( left: 0, systemBars.top, right: 0, bottom: 0);
            return insets;
        });

        mAuth = FirebaseAuth.getInstance();

        EditText etEmail = findViewById(R.id.et_forgot_email);
        MaterialButton btnSendReset = findViewById(R.id.btn_send_reset);

        findViewById(R.id.btn_forgot_back).setOnClickListener( View v -> finish());

        btnSendReset.setOnClickListener( View v -> {
            String email = etEmail.getText().toString().trim();

            if (email.isEmpty()) {
                Toast.makeText( context: this, text: "Please enter your email", Toast.LENGTH_SHORT).show();
                return;
            }
```

```
        mAuth = FirebaseAuth.getInstance();

        EditText etEmail = findViewById(R.id.et_forgot_email);
        MaterialButton btnSendReset = findViewById(R.id.btn_send_reset);

        findViewById(R.id.btn_forgot_back).setOnClickListener( View v -> finish());

        btnSendReset.setOnClickListener( View v -> {
            String email = etEmail.getText().toString().trim();

            if (email.isEmpty()) {
                Toast.makeText( context: this, text: "Please enter your email", Toast.LENGTH_SHORT).show();
                return;
            }

            btnSendReset.setEnabled(false);
            btnSendReset.setText("Sending...");

            // Firebase Magic happens here
            mAuth.sendPasswordResetEmail(email).addOnCompleteListener( Task<Void> task -> {
                if (task.isSuccessful()) {
                    Toast.makeText( context: this, text: "Check your email for the reset link!", Toast.LENGTH_LONG).show();
                    finish(); // Close screen and go back to login
                } else {
                    Toast.makeText( context: this, text: "Error: " + task.getException().getMessage(), Toast.LENGTH_LONG).show();
                    btnSendReset.setEnabled(true);
                    btnSendReset.setText("Send Reset Link");
                }
            });
        });
    }
}
```

6. Results and Evaluation

The Casio Cafe M-Commerce application has been thoroughly tested for functionality, performance, usability, and security. Functional testing confirmed that all essential features, including user authentication, menu browsing, beverage customization, cart management, order placement, payment integration, real-time notifications, and order tracking, function as expected. Both customer-facing and staff-facing functionalities, including the cafe staff dashboard and administrative panel, have been validated to ensure proper operation.

Usability testing involved real users interacting with the app, including customers placing orders and cafe staff managing menu items and orders. The feedback received led to significant refinements in the user interface, navigation flow, and overall user experience, particularly in streamlining the checkout process and enhancing the order tracking interface.

Performance testing ensured the app handles multiple concurrent users browsing the menu, placing orders, and tracking deliveries without significant lag. However, some minor performance issues were observed on emulated devices when handling image-heavy menu displays, which may be due to resource constraints in the emulator environment. Testing on physical devices showed smoother performance and better responsiveness.

Security testing validated the implementation of secure authentication with email verification, encrypted payment transactions, and proper data validation. Further penetration testing and comprehensive security assessments are planned for future iterations to strengthen the application's security posture.

Although the project has successfully met its primary objectives of creating a functional M-Commerce platform for Casio Cafe, further validation through real-world deployment and continuous user feedback will help refine the application. Additional testing, such as stress testing for scalability during peak ordering hours, load balancing assessments, and comprehensive API security evaluations, is required to enhance the application's reliability and robustness for production deployment.

7. Future Work

For future enhancements, the Casio Cafe M-Commerce application can be expanded with the following features:

- Implement AI-powered beverage recommendations based on user preferences, order history, and popular choices.
- Expand to an iOS version to reach a wider customer base across different mobile platforms.
- Integrate loyalty and rewards programs to encourage repeat customers and enhance customer retention.
- Implement advanced analytics and machine learning for inventory management and demand forecasting.
- Add social media integration allowing customers to share their favorite beverages and experiences.
- Implement subscription-based services for regular customers (e.g., daily coffee subscriptions).
- Include every feature update that modern M-Commerce applications should have by 2026, ensuring the platform remains competitive and user-centric.

8. Conclusions

The Casio Cafe M-Commerce application successfully provides a comprehensive digital solution for modernizing the cafe ordering experience in Sri Lanka. Its feature-rich platform ensures a seamless and secure transaction experience for customers and cafe staff, offering an intuitive user interface, multiple payment options, real-time order tracking, and efficient menu management features. The integration of modern technologies such as Firebase Realtime Database, Google Maps API, push notifications, and secure payment gateway processing enhances the overall ordering and operational experience, making it a convenient and reliable choice for both customers seeking quick beverage orders and cafe management looking to streamline their operations. This M-Commerce platform demonstrates the potential for digital transformation in Sri Lanka's food and beverage industry, paving the way for improved customer satisfaction and business efficiency.

9. References

UiLover. (2024). *Introduction to Web Accessibility*. YouTube. Available at: https://youtu.be/dEyGneGeAv4 (Accessed: 26 March 2026).

Chafey, D. and Ellis-Chadwick, F. (2022) *Digital Marketing: Strategy, Implementation and Practice*. 8th edn. Pearson Education.

Google Developers (2024) *Firebase Realtime Database Documentation*. Available at: https://firebase.google.com/docs/database (Accessed: 26 March 2026).

Android Developers (2024) *Google Maps SDK for Android*. Available at: https://developer.android.com/training/maps (Accessed: 26 March 2026).

Nielsen, J. (2012) *Usability Engineering*. Morgan Kaufmann.

Shneiderman, B. et al. (2017) *Designing the User Interface: Strategies for Effective Human-Computer Interaction*. 6th edn. Pearson.

Cook, T. (n.d.). *Technology should make life better and easier for people*. Apple Inc.